7 · Exp 8 — Quick sort, and comparison with other sorts

The hardest program in the lab: recursion + partition. Learn partition first; once that's solid, quickSort itself is 6 lines.

Pick pivot (last) → **Partition: small left, big right** → Pivot is now final → Recurse on left part → Recurse on right part

7.1 The idea: divide and conquer

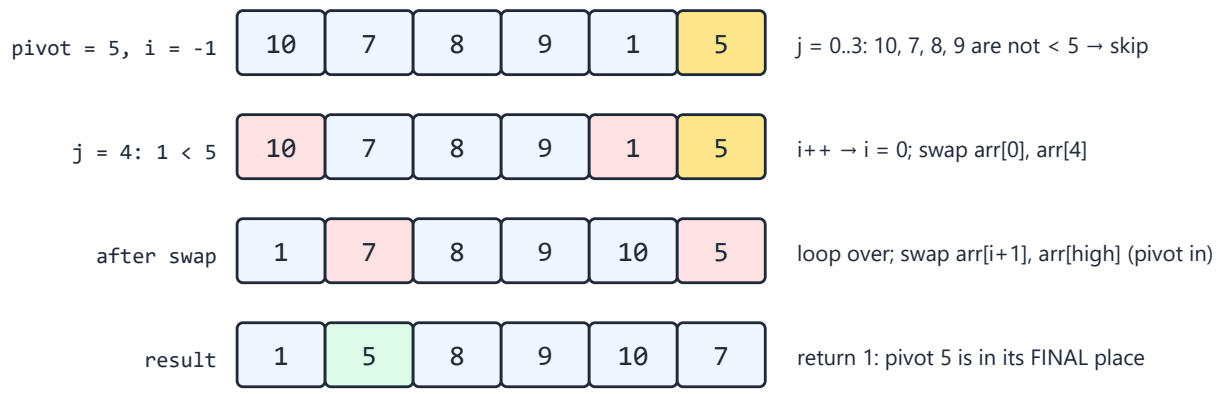
1. **Pick a pivot.** This program uses the **last** element.
2. **Partition:** rearrange so everything smaller than the pivot is on its left and everything else on its right. The pivot is then in its **final sorted position**.
3. **Recurse:** quick-sort the left part and the right part the same way. A part of 0 or 1 element is already sorted, and the recursion stops there.

7.2 Partition, step by step (learn this cold)

GUESS FIRST

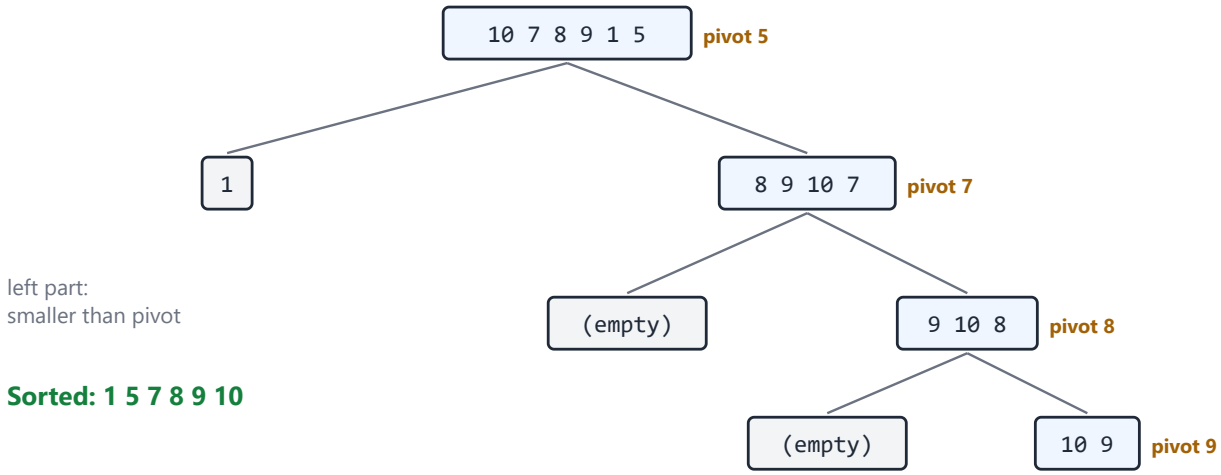
In `partitionArray`, the variable `i` starts at `low - 1`. What does `i` keep track of? (Hint: watch where small elements get swapped to.)

```
int pivot = arr[high];
int i = low - 1;                // last index of the "smaller than pivot" zone
for (int j = low; j < high; j++)
    if (arr[j] < pivot)         // small one found: grow the zone, swap it in
    {
        i++;
        swapElements(arr[i], arr[j]);
    }
swapElements(arr[i + 1], arr[high]);   // pivot goes right after the zone
return i + 1;                          // pivot's final index
```



yellow = pivot, red = the two cells being swapped, green = pivot placed

First partition of the manual's example 10 7 8 9 1 5. Same result as the manual's "Pass 1".



The whole recursion. Each box is one call to partition; the manual's Passes 1–4 are these four pivots.

WHAT `INT &A` DOES IN SWAPELEMENTS

`swapElements(arr[i], arr[j])` must change the array itself. `int &a` makes `a` **another name** for `arr[i]` (a reference, chapter 1.5). With plain `int a`, the function would swap two copies and the array would not change.

7.3 The program (corrected)

Same VLA fix as Exp 7 (`int arr[100]`). Partition and recursion are exactly the manual's.

E08_QUICK_SORT.CPP

```cpp
#include <iostream>
using namespace std;

// Function to swap two elements (& = work on the caller's variables)
void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

// Function to partition the array around the last element
int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];                  // last element as pivot
    int i = low - 1;                        // end of the "smaller than pivot" zone

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    }
    swapElements(arr[i + 1], arr[high]);    // pivot goes just after the small zone
    return i + 1;                           // pivot's final index
}
```

```cpp
// Function to implement Quick Sort
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);         // sort left part
        quickSort(arr, pi + 1, high);        // sort right part
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    int arr[100];                            // FIX: was int arr[n] (not standard C++)

    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array using Quick Sort: ";
    displayArray(arr, n);

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
Original Array: 10 7 8 9 1 5
Sorted Array using Quick Sort: 1 5 7 8 9 10
```

With duplicates and negatives:

SCREEN (REAL RUN)

```
Enter number of elements: 8
Enter elements of the array:
4 4 1 9 0 4 -3 7
Original Array: 4 4 1 9 0 4 -3 7
Sorted Array using Quick Sort: -3 0 1 4 4 4 7 9
```

7.4 The missing half: comparing with other sorts

The aim says "compare its performance with other sorting techniques". The manual's algorithm (steps 11–12) and explanation table list `bubbleSort()`, `selectionSort()`, `insertionSort()`, `copyArray()` and `clock()`, but **none of them are in the printed code**. Here is the full program:

- Every sort gets its **own copy** of the same input (`copyArray`). Sorting the already-sorted output again wouldn't be a fair test.
- Timing **6 numbers** gives 0 ms for everything, so part 2 times 10 000 random numbers. `srand(1)` makes the "random" data the same on every run.
- `clock()` counts processor ticks; `(end - start) / CLOCKS_PER_SEC` converts to seconds, and × 1000 to milliseconds.

E08_SORT_COMPARE.CPP

```cpp
#include <iostream>
#include <cstdlib>     // rand, srand
#include <ctime>       // clock, CLOCKS_PER_SEC
using namespace std;

const int BIG = 10000;                      // size used for the timing test

void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
    }
}

// Bubble Sort: neighbours swap until the largest "bubbles" to the end
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
            {
                swapElements(arr[j], arr[j + 1]);
                swapped = true;
            }
        if (!swapped)                       // no swap in a pass = already sorted
            break;
    }
}

// Selection Sort: pick the smallest of the rest, put it at position i
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swapElements(arr[i], arr[minIndex]);
    }
}

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void copyArray(int source[], int dest[], int n)
{
    for (int i = 0; i < n; i++)
        dest[i] = source[i];
}

void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

```
int original[BIG], work[BIG];               // global: too big for the stack

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> original[i];

    // Part 1: every algorithm sorts the SAME input (each gets its own copy)
    copyArray(original, work, n);  quickSort(work, 0, n - 1);
    cout << "Quick Sort     : ";  displayArray(work, n);
    copyArray(original, work, n);  bubbleSort(work, n);
    cout << "Bubble Sort    : ";  displayArray(work, n);
    copyArray(original, work, n);  selectionSort(work, n);
    cout << "Selection Sort : ";  displayArray(work, n);
    copyArray(original, work, n);  insertionSort(work, n);
    cout << "Insertion Sort : ";  displayArray(work, n);

    // Part 2: timing needs a big array - 6 numbers sort in ~0 ms
    srand(1);                                // fixed seed: same "random" data every run
    for (int i = 0; i < BIG; i++)
        original[i] = rand() % 100000;

    cout << "\nTime to sort " << BIG << " random numbers:" << endl;
    clock_t start, end;

    copyArray(original, work, BIG);
    start = clock();  quickSort(work, 0, BIG - 1);  end = clock();
    cout << "Quick Sort     : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  bubbleSort(work, BIG);  end = clock();
    cout << "Bubble Sort    : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  selectionSort(work, BIG);  end = clock();
    cout << "Selection Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  insertionSort(work, BIG);  end = clock();
    cout << "Insertion Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    return 0;
}
```

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
```

```
Quick Sort     : 1 5 7 8 9 10
Bubble Sort    : 1 5 7 8 9 10
Selection Sort : 1 5 7 8 9 10
Insertion Sort : 1 5 7 8 9 10

Time to sort 10000 random numbers:
Quick Sort     : 1 ms
Bubble Sort    : 114 ms
Selection Sort : 53 ms
Insertion Sort : 37 ms
```

The times vary from PC to PC, but the **ranking** is what the experiment shows: quick sort is far faster than the three $O(n^2)$ sorts on large random input.

Algorithm	Best	Average	Worst	Extra space	Stable?
Quick sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n) avg (recursion)	No
Bubble sort (with early exit)	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge sort (assignment)	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

QUICK SORT'S WORST CASE, IN THIS PROGRAM

With the **last element as pivot**, an **already sorted** array is the worst case: every pivot is the largest, so each partition peels off just one element. That gives $n(n-1)/2$ comparisons (15 for $n = 6$; tested in 7.6). Fix: pick the middle (or a random) element as pivot.

WRITE IN YOUR RECORD

Aim: To sort an integer array using quick sort and compare its performance with bubble, selection and insertion sort.

Algorithm: quickSort(low, high): if low < high → pi = partition(low, high) → quickSort(low, pi−1) → quickSort(pi+1, high). partition: pivot = arr[high], i = low−1; for j = low…high−1: if arr[j] < pivot, i++ and swap arr[i], arr[j]; finally swap arr[i+1], arr[high]; return i+1. Comparison: copy the same input for each sort, time each with clock().

Complexity: best/average O(n log n), worst $O(n^2)$ (sorted input with last-element pivot); space O(log n) average for the recursion, O(n) worst.

Result: 10 7 8 9 1 5 → 1 5 7 8 9 10. All four sorts gave the same output. On 10 000 random numbers quick sort took about 1 ms; bubble, selection and insertion took tens to over a hundred ms (bubble slowest).

7.5 Viva questions

Question	Answer
What is divide and conquer?	Split into smaller subproblems, solve each (recursively), combine. Quick sort splits by a pivot.
What is a pivot?	The element used to split; after partition it sits at its final sorted index.
Why `low < high` in quickSort?	A part with 0 or 1 element is sorted. This is the recursion's stopping condition (base case).
Why O(n log n) on average?	Each level of recursion does O(n) partition work, and balanced splits give about $\log_2 n$ levels.
Worst case and when?	$O(n^2)$, when the pivot is always the smallest or largest (sorted / reverse-sorted input with last-element pivot).
Space complexity?	In-place, but the recursion stack uses O(log n) on average and O(n) in the worst case.
Is quick sort stable?	No. Long-distance swaps can reorder equal elements.
Why copy the array before each sort in the comparison?	So every algorithm sorts the same unsorted input; a fair test.
Why does `swapElements` use `&`?	To swap the actual array elements, not copies.

7.6 If the examiner changes the question

"Separate functions for input, partitioning, sorting and display" is the program above. The rest (7 checks, all pass):

Descending order (one character in partition):

```
// Descending: the ONLY change is  arr[j] < pivot  ->  arr[j] > pivot
int partitionDesc(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] > pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortDesc(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionDesc(arr, low, high);
        quickSortDesc(arr, low, pi - 1);
```

```
        quickSortDesc(arr, pi + 1, high);
    }
}
```

First element as pivot: swap it to the end, then partition exactly as before (`quickSort` stays the same, just calling `partitionFirst`):

```
// First element as pivot: swap it to the end, then partition exactly as before
int partitionFirst(int arr[], int low, int high)
{
    swapElements(arr[low], arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}
```

Middle element as pivot (the worst-case fix):

```
// Middle element as pivot: same trick (sorted input no longer gives the worst case)
int partitionMiddle(int arr[], int low, int high)
{
    int mid = low + (high - low) / 2;
    swapElements(arr[mid], arr[high]);
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortMiddle(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionMiddle(arr, low, high);
        quickSortMiddle(arr, low, pi - 1);
        quickSortMiddle(arr, pi + 1, high);
    }
}
```

Count comparisons and swaps. On the manual's example this gives 11 comparisons and 5 swaps; on a sorted array of 6, 15 comparisons (the worst case). Run it on sorted, reverse-sorted and random input for the "compare performance" item.

```
// Global counters: every function can add to them; reset to 0 before sorting
int comparisons = 0, swaps = 0;

int partitionCount(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        comparisons++;
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
            swaps++;
        }
    }
    swapElements(arr[i + 1], arr[high]);
    swaps++;
    return i + 1;
}

void quickSortCount(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionCount(arr, low, high);
        quickSortCount(arr, low, pi - 1);
        quickSortCount(arr, pi + 1, high);
    }
}
```

Merge sort (for "compare quick sort with merge sort"; time it with `clock()` as in 7.4):

```
// Merge Sort: split in half, sort each half, merge the two sorted halves
void merge(int arr[], int low, int mid, int high)
{
    int temp[100];                          // holds the merged result
    int i = low, j = mid + 1, k = 0;
    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= mid)                        // leftovers of the left half
        temp[k++] = arr[i++];
    while (j <= high)                       // leftovers of the right half
        temp[k++] = arr[j++];
    for (int t = 0; t < k; t++)             // copy back
        arr[low + t] = temp[t];
}

void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;
```

```
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
```

CHECK 7.1

Partition `4 9 2 7 5` (pivot 5). Track i and the array after each swap. What index does partition return?

CHECK 7.2

Without looking, write `quickSort(int arr[], int low, int high)`. Six lines.

Answers

Check 7.1: pivot 5, i = −1. j = 0: 4 < 5 → i = 0, swap(arr[0], arr[0]) → `4 9 2 7 5`. j = 1: 9 no. j = 2: 2 < 5 → i = 1, swap(arr[1], arr[2]) → `4 2 9 7 5`. j = 3: 7 no. End: swap(arr[2], arr[4]) → `4 2 5 7 9`. Returns **2**.

Check 7.2: `void quickSort(int arr[], int low, int high) { if (low < high) { int pi = partitionArray(arr, low, high); quickSort(arr, low, pi - 1); quickSort(arr, pi + 1, high); } }`